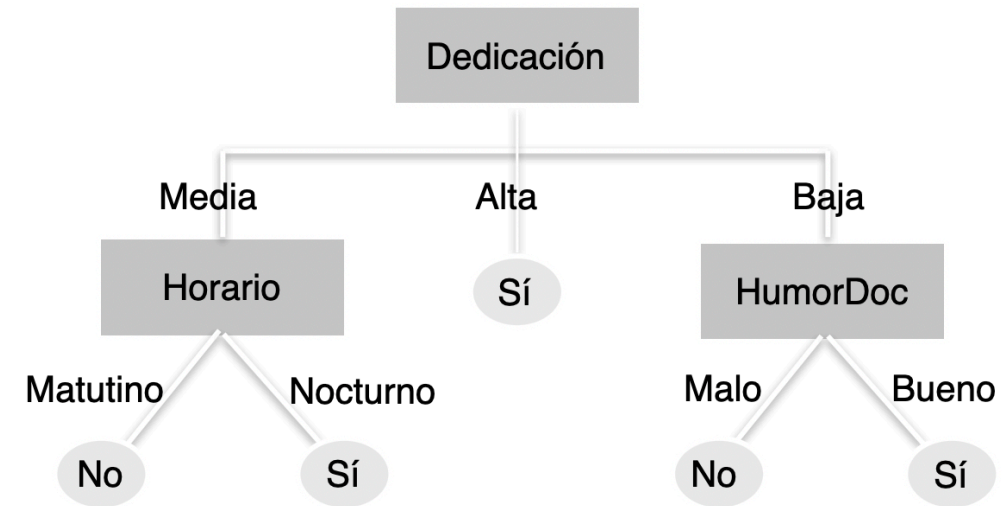
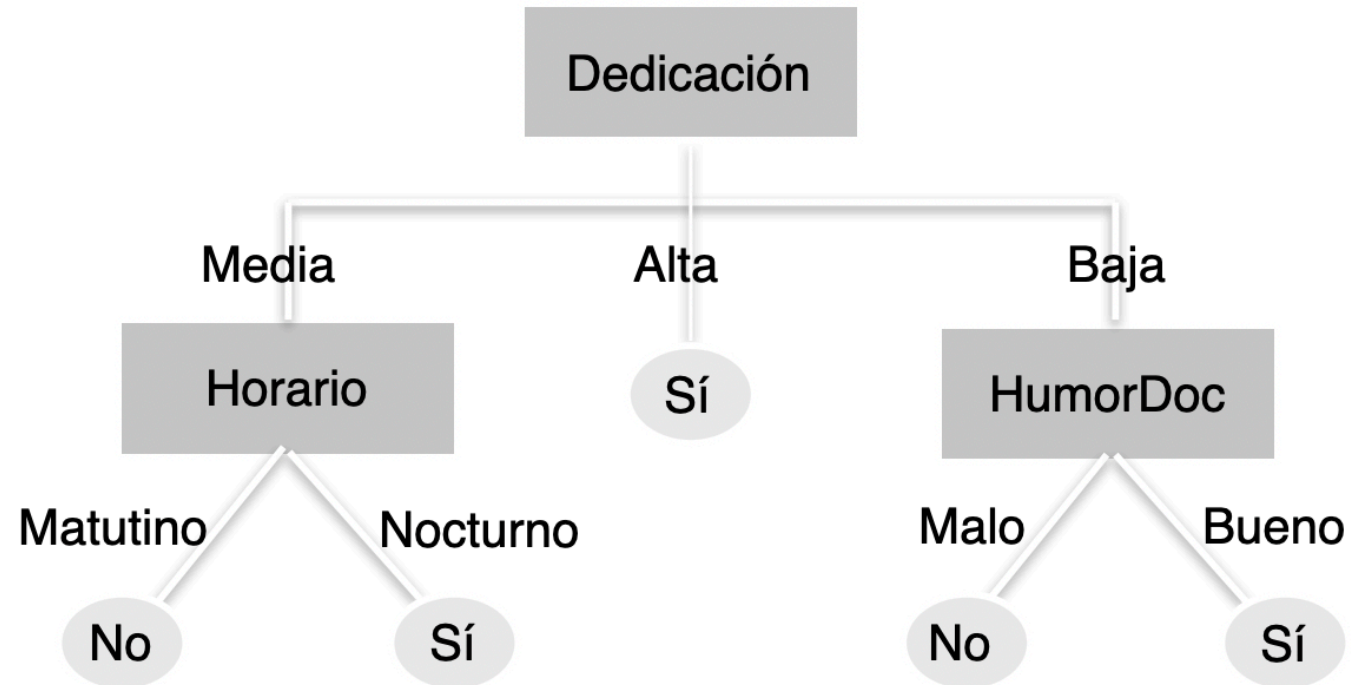


Árboles de decisión

- En cada nodo se asocia con un atributo
- Las ramas se etiquetan con sus posibles valores
- Para clasificar:
 - se comienza por la raíz
 - en cada paso se evalúa al atributo marcado por el nodo y se continúa por la rama asociada al valor de la instancia
 - el proceso se repite hasta llegar a una hoja, donde se obtiene la clasificación



Árboles de decisión

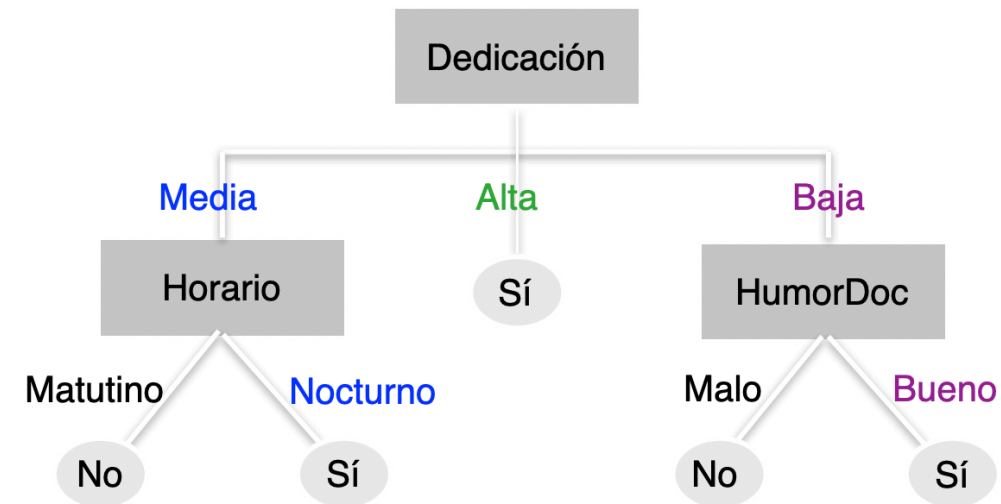


- ¿Cómo son clasificadas las siguientes instancias?
 - [$Ded = Media, Dif = Alta, Hor = Noc, Hum = Alta, HDoc = Malo$]
 - [$Ded = Baja, Dif = Alta, Hor = Noc, Hum = Alta, HDoc = Bueno$]

Árboles de decisión

- Cada rama del árbol es una restricción sobre los atributos expresada como una conjunción
- El árbol es una disyunción de las restricciones representadas por sus ramas positivas

$$\begin{aligned}Dedic &= \textit{Media} \wedge \textit{Horario} = \textit{Nocturno} \\ &\vee \\Dedic &= \textit{Alta} \\ &\vee \\Dedic &= \textit{Baja} \wedge \textit{HumorDoc} = \textit{Bueno}\end{aligned}$$



- ¿Cuántas hipótesis tiene el espacio H ?

Árboles de decisión

- Las instancias se representan como un conjunto de parejas atributo-valor
- La función objetivo toma valores discretos
- El conjunto de entrenamiento puede cometer errores
- Las instancias de entrenamiento pueden tener atributos incompletos

Algoritmo ID3

Algoritmo ID3 (*Atributos*, *Ejemplos*):

- Crear una nodo raíz
- Si todos los ejemplos tienen el mismo valor $\rightarrow$ etiquetar con ese valor
- Si $Atributos = \emptyset \rightarrow$ etiquetar con el valor más común de *Ejemplos*
- En caso contrario:
 - Se etiqueta con un atributo $A \in Atributos$ (que mejor clasifica los ejemplos)
 - Para cada valor posible de A , v_i :
 - Genero una rama etiquetada con v_i
 - $Ejemplos_{v_i} = \{x \in Ejemplos / x[A] = v_i\}$
 - Si $Ejemplos_{v_i} = \emptyset \rightarrow$ etiquetar con el valor más común de *Ejemplos*
 - En caso contrario $\rightarrow ID3(Ejemplos_{v_i}, Atributos - A)$

Algoritmo ID3

- Construye un árbol desde la raíz hacia las hojas (*top down*)
- Cuando todas las instancias son iguales, no tengo atributos o me quedo sin ejemplos para una rama, se crea una hoja
- En cada paso se elige un atributo y se genera una rama por cada uno de sus posibles valores
- El proceso se repite recursivamente en cada una de las ramas, reduciendo los ejemplos a considerar en cada paso
- Nunca se vuelve atrás una decisión; nunca se verifica que el atributo seleccionado en cada paso sea realmente el mejor

Algoritmo ID3 - Pedro

#	Dedicación	Dificultad	Horario	Humedad	Humor Doc	$c(x)$
1	Alta	Alta	Nocturno	Media	Bueno	Sí
2	Baja	Media	Matutino	Alta	Malo	No
3	Media	Alta	Nocturno	Media	Malo	Sí
4	Media	Alta	Matutino	Alta	Bueno	No

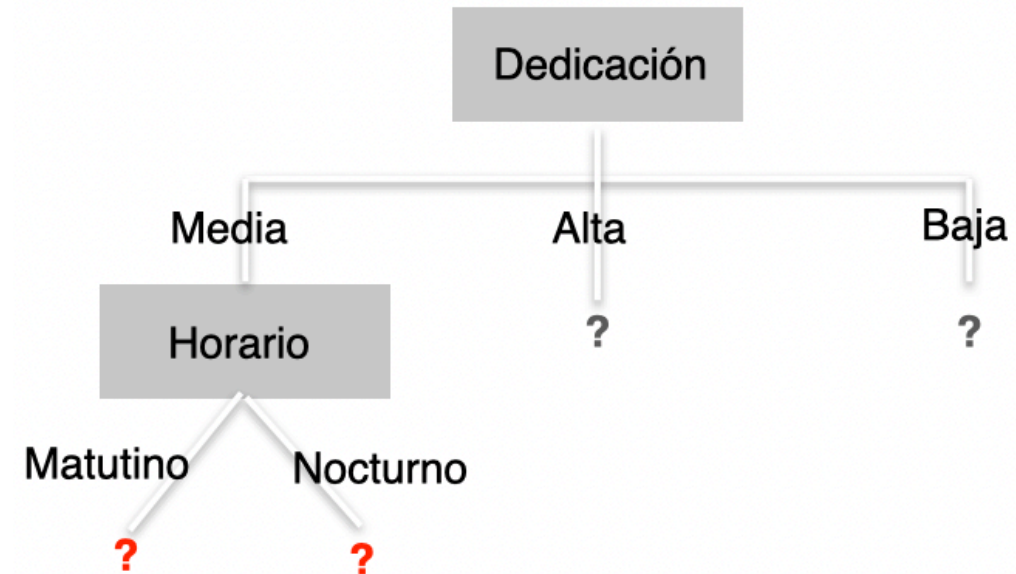
- Supongamos que un óraculo determina *Dedicación* como mejor atributo
- Se crean tres ramas: *Media*, *Alta* y *Baja*
- Se aplica ID3 recursivamente en cada rama



Algoritmo ID3 - Pedro

#	Dedicación	Dificultad	Horario	Humedad	Humor Doc	$c(x)$
3	Media	Alta	Nocturno	Media	Malo	Sí
4	Media	Alta	Matutino	Alta	Bueno	No

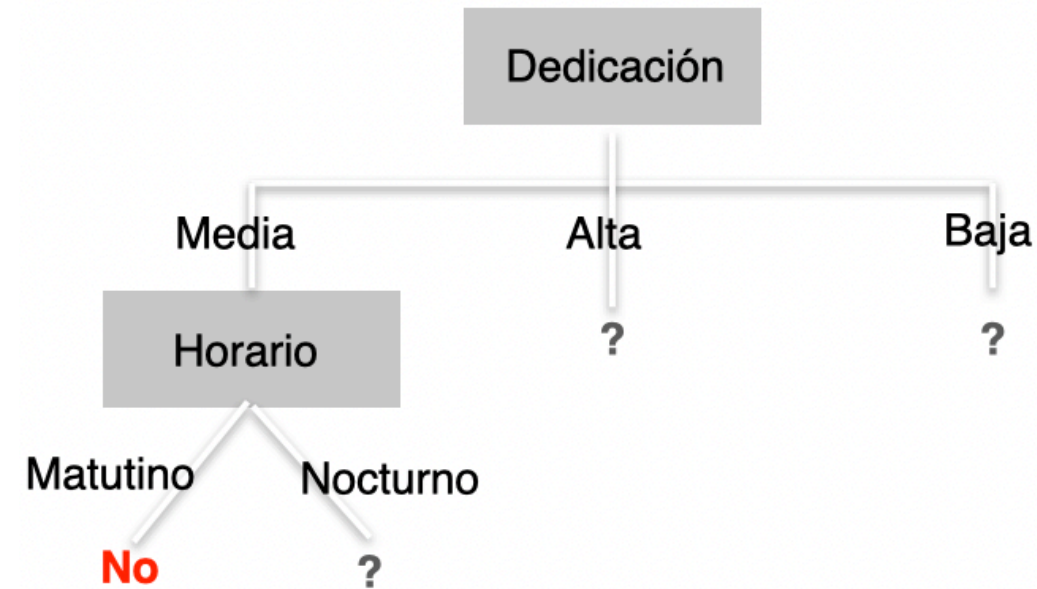
- Aplicamos recursivamente ID3 en la primera rama:
 - Los ejemplos cumplen $Dedic. = Media$
 - Se elimina el atributo *Dedicación*
- El óraculo elige *Horario* como mejor atributo
- Se crean dos ramas: *Matutino* y *Nocturno*



Algoritmo ID3 - Pedro

#	Dedicación	Dificultad	Horario	Humedad	Humor Doc	$c(x)$
4	Media	Alta	Matutino	Alta	Bueno	No

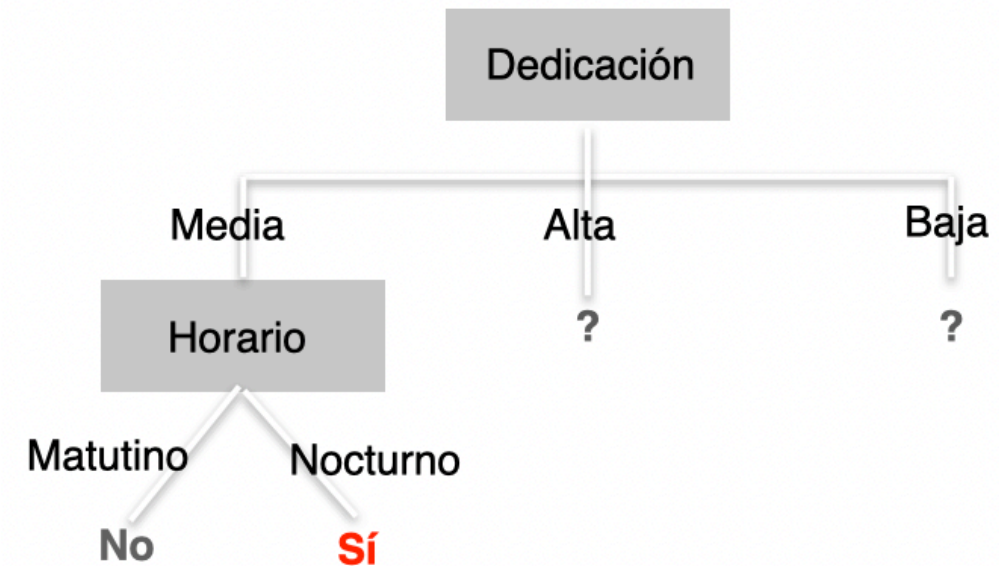
- Aplicamos recursivamente ID3 en la primera rama:
 - Los ejemplos cumplen $Horario = Mat.$
 - Se elimina el atributo $Horario$
- Como todas las instancias son negativas, se etiqueta con No



Algoritmo ID3 - Pedro

#	Dedicación	Dificultad	Horario	Humedad	Humor Doc	$c(x)$
3	Media	Alta	Nocturno	Media	Malo	Sí

- Aplicamos recursivamente ID3 en la segunda rama:
 - Los ejemplos cumplen $Horario = Noct.$
 - Se elimina el atributo $Horario$
- Como todas las instancias son positivas, se etiqueta con $Sí$
- ¿Y si $Horario$ tuviese un valor $Vespertino$?



Selección de atributo

- Vamos a utilizar una medida de cuánto un atributo separa a los ejemplos según su clasificación:

Entropía

Sea S un conjunto de ejemplos, f una función objetivo que toma los valores $v_1 \dots v_n$

$$Entropía(S) = - \sum_{v_i} p_{v_i} \log_2(p_{v_i})$$

donde p_{v_i} es la proporción de ejemplos $x \in S, f(x) = c_i$

- En particular, para funciones booleanas:

$$Entropía(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

Selección de atributo

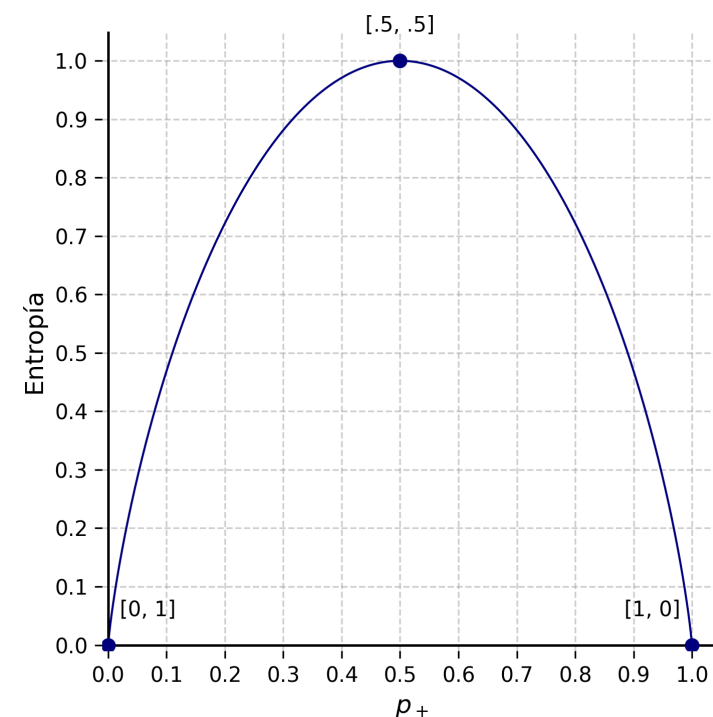
- La entropía es la cantidad mínima de bits que en promedio se requiere para codificar los elementos de S
- Mide la homogeneidad de los datos: cuanto más homogéneos sean, menor será su entropía
- Ejemplos:

$$\circ \text{Entropía}([9+, 0-]) = -\frac{9}{9}\log_2\left(\frac{9}{9}\right) - \frac{0}{9}\log_2\left(\frac{0}{9}\right) = 0$$

$$\circ \text{Entropía}([9+, 9-]) = -\frac{1}{2}\log_2\left(\frac{1}{2}\right) - \frac{1}{2}\log_2\left(\frac{1}{2}\right) = 1$$

$$\circ \text{Entropía}([9+, 5-]) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940$$

➡ Tomamos $\frac{0}{n}\log_2 \frac{0}{n} = 0$



Selección de atributo

- Definimos la ganancia de información de un atributo A sobre una muestra S :

Ganancia

$$Ganancia(S, A) = Entropía(S) - \sum_{v \in Valores(A)} \frac{|S_{A=v}|}{|S|} Entropía(S_{A=v})$$

- Buscamos medir la reducción de entropía al particionar por A
- Cuanto más homogéneos sean los conjuntos, menor su entropía y mayor su ganancia
- $Ganancia$ es el número de bits que se ahorran al saber el valor de A

Selección de atributo - Pedro

#	Dedicación	Dificultad	Horario	Humedad	Humor Doc	$c(x)$
1	Alta	Alta	Nocturno	Media	Bueno	Sí
2	Baja	Media	Matutino	Alta	Malo	No
3	Media	Alta	Nocturno	Media	Malo	Sí
4	Media	Alta	Matutino	Alta	Bueno	No

- La entropía de S para este conjunto es:

$$Entropía([2+, 2-]) = -\frac{1}{2}\log_2\left(\frac{1}{2}\right) - \frac{1}{2}\log_2\left(\frac{1}{2}\right) = 1$$

- ¿Nos convendría particionar por *HumorDoc* en lugar de *Dedicación*?

Selección de atributo - Pedro

- La ganancia de particionar por *Dedicación* es:

$$S_{Ded=Alta} \leftarrow [1+, 0-] \quad S_{Ded=Media} \leftarrow [1+, 1-] \quad S_{Ded=Baja} \leftarrow [0+, 1-]$$

$$Entropía(S_{Ded=Alta}) = Entropía(S_{Ded=Baja}) = 0 \quad Entropía(S_{Ded=Media}) = 1$$

$$\begin{aligned} Ganancia(S, Ded) &= 1 - \frac{1}{4}Ent(S_{Ded=Alta}) - \frac{2}{4}Ent(S_{Ded=Media}) - \frac{1}{4}Ent(S_{Ded=Baja}) \\ &= 1 - \frac{1}{4}0 - \frac{1}{2}1 - \frac{1}{4}0 \\ &= \frac{1}{2} \end{aligned}$$

Selección de atributo - Pedro

- Y por *HumorDoc*:

$$S_{HDoc=Bueno} \leftarrow [1+, 1-] \quad S_{HDoc=Malo} \leftarrow [1+, 1-]$$

$$Entropía(S_{HDoc=Bueno}) = Entropía(S_{HDoc=Malo}) = 1$$

$$\begin{aligned} Ganancia(S, HDoc) &= 1 - \frac{2}{4} Ent(S_{HDoc=Bueno}) - \frac{2}{4} Ent(S_{HDoc=Malo}) \\ &= 1 - \frac{1}{2} \cdot 1 - \frac{1}{2} \cdot 1 \\ &= 0 \end{aligned}$$

- *HumorDoc* no es una mejor elección que *Dedicación*

Selección de atributo - Pedro

- Sin embargo, *Horario* sí lo es:

$$S_{Hor=Mat} \leftarrow [0+, 2-] \quad S_{Hor=Noct} \leftarrow [2+, 0-]$$

$$Entropía(S_{Hor=Mat}) = Entropía(S_{Hor=Noct}) = 0$$

$$\begin{aligned} Ganancia(S, Hor) &= 1 - \frac{2}{4} Ent(S_{Hor=Mat}) - \frac{2}{4} Ent(S_{Hor=Noct}) \\ &= 1 - \frac{1}{2} 0 - \frac{1}{2} 0 \\ &= 1 \end{aligned}$$

Búsqueda con ID3

- Como toda función discreta se puede representar con un árbol, el espacio H es completo: el concepto objetivo pertenece al espacio de hipótesis
- ID3 realiza una búsqueda en el espacio de árboles, del más sencillo al más complejo
- ID3 mantiene una única hipótesis: no hay forma de saber cuáles ni cuántos árboles equivalentes hay en el espacio
- ID3 no realiza *backtracking*: una vez elegido un atributo, nunca se pregunta si esa fue la mejor opción; esto puede conducir a una solución que es óptima local pero no globalmente
- Se utilizan todos los ejemplos en cada paso: esto evita que el algoritmo sea muy sensible al ruido, a diferencia de, por ejemplo, Candidate-elimination

Sesgo inductivo

- El algoritmo ID3 elige un árbol que (aprox.):
 - cubre los ejemplos
 - es el más simple posible (menos profundo)
 - los atributos con más ganancia de información están más cerca de la raíz
- El sesgo está en el algoritmo y no en el espacio.

Sesgo inductivo

Sesgo preferencial

El algoritmo prefiere ciertas hipótesis sobre otras.

Sesgo restrictivo

El algoritmo maneja un espacio de hipótesis incompleto.

- Por ejemplo: ID3 tiene un sesgo preferencial, mientras que en nuestro ejemplo de Candidate-elimination, uno restrictivo
- Es deseable tener únicamente sesgo preferencial: esto asegura que el concepto objetivo se encuentra en H
- El aprendizaje puede tener sesgos de ambos tipos

Sesgo inductivo

¿Por qué preferir hipótesis más simples?

Principio de la navaja de Ockham

Cuando se ofrecen dos o más explicaciones de un fenómeno, es preferible la explicación completa más simple; es decir, no deben multiplicarse las entidades sin necesidad.

[Prefiera la hipótesis más simple que se ajuste a los datos.]

William de Ockham (S.XIV)

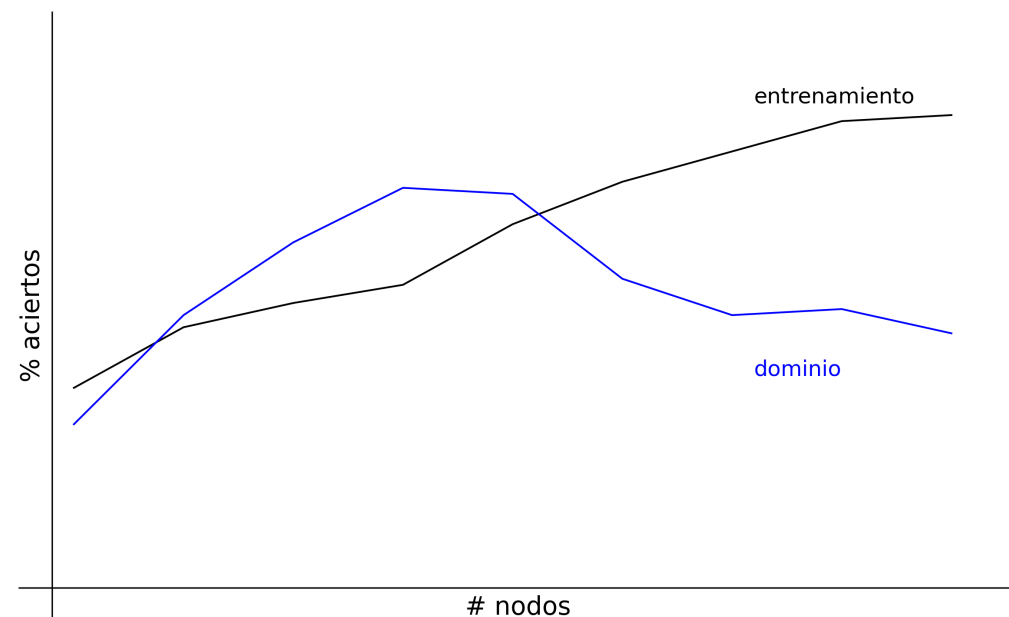
- Como hay menos hipótesis simples, es menos probable que una de ellas se ajuste a los datos “porque sí”
- Más adelante veremos una justificación bayesiana a este principio

Variantes y extensiones

- Evitar el sobreajuste al conjunto de entrenamiento.
- Manejar atributos con valores continuos.
- Utilizar medidas alternativas para la elección del atributo.
- Manejar ejemplos de entrenamiento con atributos incompletos.
- Preguntar por múltiples atributos en un nodo.
- Combinación lineal de atributos numéricos.

Evitar el sobreajuste

Sobreajuste: $h \in H$ sobreajusta a un conjunto de entrenamiento S si existe $h' \in H$, tal que h comete menos errores que h' en el conjunto S pero comete más errores en el todo el dominio D .



Evitar el sobreajuste

- El sobreajuste puede ser causado por:
 - Errores en los datos de entrenamiento: el árbol aprende a clasificar correctamente, y lo inducen a clasificar erróneamente ejemplos del dominio
 - Cantidad insuficiente de ejemplos: aparecen regularidades que no se cumplen en el dominio. En nuestro ejemplo: ¿es tan importante el horario en que se da un examen?
- ¿Cómo evitarlo?
 - Detener el crecimiento del árbol antes de que se ajuste perfectamente a los datos
 - Aplicar técnicas de podado luego de construir el árbol

Evitar el sobreajuste

¿Cómo determinamos el tamaño óptimo del árbol?

- Con un conjunto de ejemplos no utilizados durante el entrenamiento (conjunto de validación).
- Aplicando técnicas estadísticas para evaluar si es conveniente agregar/quitar nodos.
- Usando una medida de complejidad para codificar los datos y el tamaño del árbol, y parando cuando esa medida se minimiza.

Poda

- Consideramos cada nodo y lo sustituimos por la valoración más común de sus ejemplos, siempre que mejore el acierto sobre el conjunto de validación
- Se precisa un conjunto relativamente grande de ejemplos

Poda por reglas - C4.5

- Transformamos al árbol en el conjunto de reglas que representa
- Quitamos condiciones de las reglas de forma que mejoren su estimación en el conjunto de validación.
- Ordenamos las reglas de acuerdo a su porcentaje de acierto y, ante nuevas instancias, las aplicamos en ese orden

Poda por reglas - C4.5

- Al separar las ramas, se pueden cortar atributos en algunas de ellas y no en otras
- Es más fácil sacar conjunciones que quitar ramas enteras de un árbol
- Las reglas son legibles para un ser humano

Atributos con valores continuos

- ¿Que sucede si la humedad viene dada por un porcentaje?
- Se pueden generar dinámicamente cortes para esos atributos, que luego compiten como atributos discretos

		58,5%		87,5%			
Humedad	50%	67%	71%	82%	93%	95%	99%
Salva	Sí	No	No	No	Sí	Sí	Sí

- Se consideran a los nuevos atributos $Humedad_{>58,5}$ y $Humedad_{>87,5}$
- Los árboles de regresión permiten manejar los casos en donde la propio función objetivo no es discreta

Alternativas en la selección del atributo

- *Ganancia* favorece a los atributos con más valores posibles: ¿qué sucede con humedad o una fecha?
- Se puede penalizar la distribución uniforme de valores:

$$SplitInformation(S, A) = - \sum_{v \in Val(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

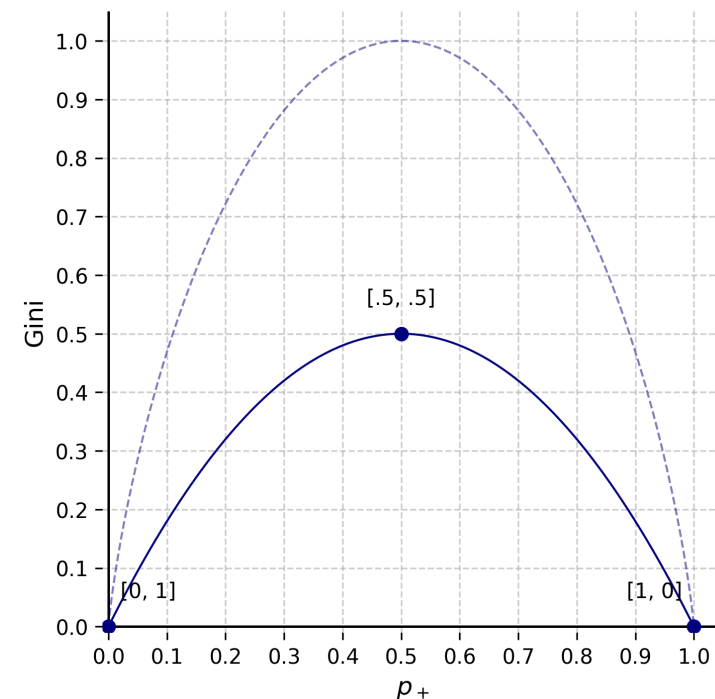
$$GainRatio(S, A) = \frac{Ganancia(S, A)}{SplitInformation(S, A)}$$

Alternativas en la selección del atributo

- Otra medida es la reducción de la impureza:

$$Gini(S) = 1 - \sum p_i^2 = 1 - p_+^2 - p_-^2$$

$$ImpurityReduction(S, A) = Gini(S) - \sum_{v \in Val(A)} \frac{|S_v|}{|S|} Gini(S_v)$$



Atributos incompletos

- A los ejemplos les pueden faltar valores en algunos de sus atributos
- Para suplirlos se utilizan los otros ejemplos que sí los tengan:
 - el valor más común de ese atributo faltante en el subconjunto de ese nodo o en todo el conjunto
 - un valor de acuerdo a cierta distribución, usualmente estimada a partir de los otros ejemplos

Otras extensiones

- Preguntar por múltiples atributos en un nodo
- Combinación lineal de atributos numéricos: se pueden hacer cortes oblicuos
- Múltiples clases en lugar de valores booleanos:
 - ¿se debe separar una clase de las otras? ¿o un grupo de ellas vs. el resto?
 - entreno un árbol por cada valor posible de la salida, pero ¿qué sucede si más de un árbol lo clasifica como de su clase?
- ¿qué sucede con la legibilidad de la solución?

Bosques

- Para atenuar la sensibilidad de un árbol al conjunto de entrenamiento, se pueden entrenar varios árboles (poco correlacionados) y ensamblarlos con votación.
- Existen varias alternativas, por ejemplo:
 - Random Forest
 - Extra Trees
 - AdaBoost

Random Forest

- Se construyen m árboles de decisión
- La clasificación se obtiene por votación
- Para cada árbol solo se toman en cuenta p atributos elegidos al azar
- Dados los ejemplos D , se crean m nuevos conjuntos D_i seleccionando al azar k elementos con repetición de forma uniforme (Bagging)
- Por ejemplo, con $D = [x_1, x_2, x_3, x_4, x_5]$, $k = |D|$:

$$D_1 = [x_1, x_2, x_2, x_5, x_5] \dots D_m = [x_3, x_3, x_3, x_4, x_5]$$

Random Forest

- El algoritmo tiene, entonces, tres hiperparámetros:
 - m : la cantidad total de árboles: 100, 500, 1000, ...
 - k : la cantidad de ejemplos: $|D|$...
 - p : la cantidad de atributos: $\sqrt{|A|}$, $\frac{|A|}{3}$...
- ¿Cómo elegimos esos valores?
- El error se puede estimar evaluando cada elemento en aquellos árboles que no lo usaron para su construcción

Extra-Trees

- Utiliza a todo el conjunto de entrenamiento
- Se seleccionan p atributos al azar para construir el árbol
- En cada nodo el atributo se elige al azar, con un punto de corte al azar

AdaBoost

- Técnica general en donde se entrenan varios clasificadores débiles
- Por ejemplo, utilizar *Decision Stumps*: árboles de decisión realmente muy pequeños, de un atributo
- Los ejemplos tienen peso; los clasificadores se entrenan sobre aquellos que son más difíciles para los anteriores aumentando su peso
- Los clasificadores resultantes se combinan, a su vez, con distintos pesos